



Lab Activity

Week 1: While Loops - Escape the Time Loop

Duration: 60 minutes — Work in pairs!

Name: _____ Partner: _____

Lab Goals

By the end of this lab, you will be able to:

- Convert **for** loops to **while** loops and vice versa
- Build interactive programs using **while** loops
- Create proper exit conditions to avoid infinite loops
- Work effectively with a partner using pair programming
- Earn your first Time Loop badges!

Today's Story

*You're trapped in a mysterious time loop! Every day repeats exactly the same way... unless you can master the **while** loop to break free. Work with your partner to solve increasingly challenging puzzles and escape!*

Pair Programming Protocol

Remember the Driver-Navigator Model:

- **Driver:** Types the code, focuses on syntax
- **Navigator:** Thinks about logic, spots errors, guides strategy
- **Switch roles every 15 minutes!** (Your teacher will announce switches)
- Both partners should understand every line of code

1 Part 1: Loop Conversion Training (20 minutes)

1.1 Exercise 1.1: For to While Conversion

Navigator reads, Driver types!

Convert this for loop to a while loop:

```
1 # Original FOR loop
2 print("Countdown with FOR loop:")
3 for num in range(5, 0, -1):
4     print(f"T-minus {num}")
5 print("Blast off!")
6
7 # YOUR WHILE LOOP VERSION:
8 print("\nCountdown with WHILE loop:")
9 # Write your code here
```

Success Criteria:

- ✓ Output matches exactly
- ✓ Uses proper initialization
- ✓ Updates the counter correctly
- ✓ Has correct exit condition

1.2 Exercise 1.2: While to For Conversion

 **Pair Programming:** Switch who types every 5 minutes!

Convert this `while` loop to a `for` loop:

```

1 # Original WHILE loop
2 power = 1
3 while power <= 1000:
4     print(f"Power level: {power}")
5     power = power * 2
6
7 # YOUR FOR LOOP VERSION:
8 # Hint: You'll need to calculate how many iterations!
9 # Write your code here
    
```

Challenge Question: Why is this conversion trickier than the first one? Discuss with your partner.

 **Checkpoint:** Show your instructor both conversions to earn your **Loop Converter** badge!

2 Part 2: The Guessing Game Challenge (25 minutes)

2.1 Exercise 2.1: Basic Number Guesser

Work together to build a number guessing game!

Partner Activity: Navigator picks a secret number (1-10) and writes it down. Driver codes the game!

```

1 # Basic Number Guessing Game
2 # Navigator: Pick a secret number between 1 and 10
3 # Write it here (hide from driver!): _____
4
5 # Initialize your variables
6 secret = 7 # Driver: Change this to test your code
7 guess = 0 # Start with an impossible guess
8 tries = 0 # Count the attempts
9
10 print("=== TIME LOOP CHALLENGE ===")
11 print("I'm thinking of a number between 1 and 10")
12 print("Guess it to escape this day!\n")
13
14 # YOUR WHILE LOOP HERE
15 # Remember to:
16 # - Get input from user
17 # - Convert to integer
    
```

```

18 # - Give "higher" or "lower" feedback
19 # - Count tries
20 # - Update the guess variable!
21
22 # After the loop, print success message
23 print(f"Correct! You escaped in {tries} tries!")

```

2.2 Exercise 2.2: Enhanced Guessing Game

 **Pair Programming:** Switch who types every 5 minutes!

Add these features to your game:

1. **Limited Attempts:** Player only gets 5 tries
2. **Time Pressure:** Each guess "uses" time (subtract from a `time_left` variable)
3. **Replay Option:** Ask if they want to play again

```

1 # Enhanced Guessing Game with Limited Attempts
2 play_again = "yes"
3
4 while play_again.lower() == "yes":
5     # Game setup
6     secret = 5 # Change this to test
7     guess = 0
8     tries = 0
9     max_tries = 5
10    time_left = 60 # seconds
11
12    print("\n=== ENHANCED TIME LOOP CHALLENGE ===")
13    print(f"You have {max_tries} attempts and {time_left} seconds!")
14    print("Each guess uses 10 seconds of time.\n")
15
16    # YOUR WHILE LOOP HERE
17    # Hint: Your condition needs to check BOTH:
18    # - guess != secret
19    # - tries < max_tries
20    # - time_left > 0
21
22    # while ____:
23    #     _____
24    #     _____
25    #
26    #     # Get guess and give feedback
27    #     ...
28
29    # After loop: Check if they won or lost
30    # if guess == secret:
31    #     print("You escaped!")
32    # else:
33    #     print("Still trapped in the time loop!")
34
35    # Ask to play again
36    play_again = input("\nPlay again? (yes/no): ")
37
38 print("Thanks for playing! You've escaped the time loop!")

```

 **Checkpoint:** Complete both games to earn your **Game Loop Master** badge!

3 Part 3: Debug Detective (10 minutes)

3.1 Exercise 3.1: Find and Fix the Infinite Loops

Work together to debug these broken time loops!

BUG 1: Why does this create an infinite loop?

```

1 # Broken countdown - creates infinite loop!
2 count = 10
3 while count != 0:
4     print(f"Counting: {count}")
5     count = count - 3
6 print("Done!")
7
8 # YOUR FIX: (write the corrected code below)
9 # Hint: What values will count have? 10, 7, 4, 1, -2...
10 # Will it ever equal exactly 0?

```

BUG 2: This should print even numbers up to 10

```

1 # Broken code - infinite loop!
2 num = 0
3 while num < 10:
4     if num % 2 == 0:
5         print(num)
6     # What's missing here?
7
8 # YOUR FIX: (write the corrected code below)
9 # Hint: How does num change? Or... does it?

```

BUG 3: This should get valid age (0-120)

```

1 # Broken age checker
2 age = -1
3 while age < 0 or age > 120:
4     user_input = input("Enter your age: ")
5     # What's wrong here?
6     print("Invalid age, try again!")
7 print(f"Your age is {age}")
8
9 # YOUR FIX: (write the corrected code below)
10 # Hint: Look at the age variable. When does it change?

```

✔ **Checkpoint:** Fix all three bugs to earn your **Debug Detective** badge!

4 Part 4: Creative Challenge (5 minutes)

4.1 Exercise 4.1: Design Your Own Time Loop

If time permits, create your own creative while loop program!

Ideas:

- 🎵 Music player that repeats until user says "stop"
- 🏃 Fitness counter that tracks exercises
- 📅 Todo list that keeps asking for tasks
- ✍️ Pattern generator that creates ASCII art

```

1 # Create your own while loop program!
2 # Must include:
3 # - Proper initialization
4 # - Clear exit condition
5 # - User interaction
6 # - Creative theme
7
8 # Example starter for a fitness counter:
9 # exercise_count = 0
10 # continue_workout = "yes"
11 #
12 # while continue_workout == "yes":
13 #     exercise = input("What exercise did you do? ")
14 #     exercise_count = exercise_count + 1
15 #     print(f"Great! That's {exercise_count} exercises!")
16 #     continue_workout = input("Do another? (yes/no): ")
17 #
18 # print(f"Workout complete! You did {exercise_count} exercises!")
19
20 # Your creative code here:

```

5 Lab Reflection

5.1 With Your Partner, Discuss:

1. When would you choose a `while` loop over a `for` loop?
2. What was the trickiest bug you found today?
3. How did pair programming help you solve problems?

5.2 Memory Diagram Challenge

Draw a memory diagram for this code:

```

1 attempts = 0
2 password = ""
3 while password != "secret":
4     password = input("Enter password: ")
5     attempts = attempts + 1
6 print(f"Success in {attempts} tries!")

```

Draw your memory diagram here

6 Today's Achievements

Badges Available

- ☐ **Loop Converter** - Successfully convert between loop types
- ☐ **Game Loop Master** - Build working guessing games
- ☐ **Debug Detective** - Fix all infinite loops
- ☐ **Time Keeper** - Complete lab within time limit
- ☐ **Team Player** - Effective pair programming

Bonus Challenges (Optional)

1. Make the guessing game give "hot/cold" hints instead of "higher/lower"
2. Add a scoring system based on number of guesses
3. Create a two-player version where players take turns

7 Exit Ticket

Quick Check

Fill in the blanks:

1. A while loop needs three parts:
 - _____ (before the loop)
 - _____ (in the while statement)
 - _____ (inside the loop body)
2. True or False:
 - While loops always run at least once: _____
 - You can convert any for loop to a while loop: _____
 - While loops are better for user input: _____

8 Optional Home Extension

💡 **Tip:** Want more practice? Try these at home:

1. Create a “Virtual Pet” that needs feeding (while hungry)
2. Build a calculator that keeps running until user types ”quit”
3. Make a story that continues based on user choices

Remember: You’re building the skills to escape any time loop - in code and in life! 🐼

😊 **Great work today! You’re mastering the while loop!**

*Next Lab Preview: Loop Control with **break** and **continue** - Master the emergency exits!*